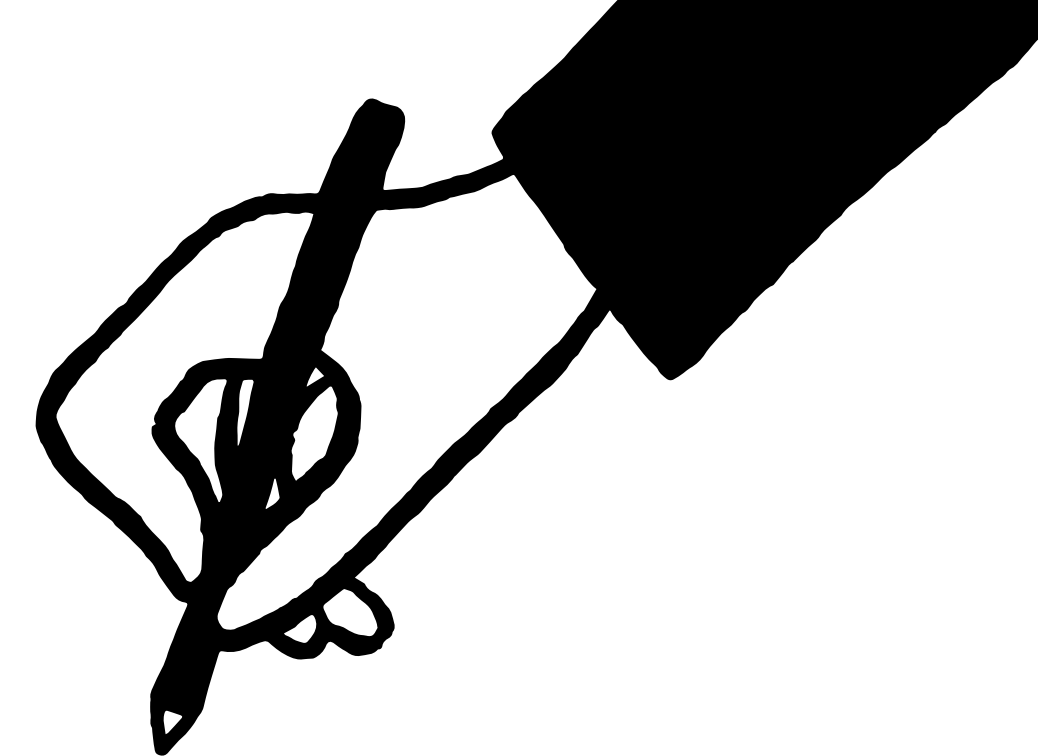




ФАКУЛТЕТ ЗА ИНФОРМАТИЧКИ НАУКИ
И КОМПЈУТЕРСКО ИНЖЕНЕРСТВО



Routing

Chapter 7

Advanced Web Design



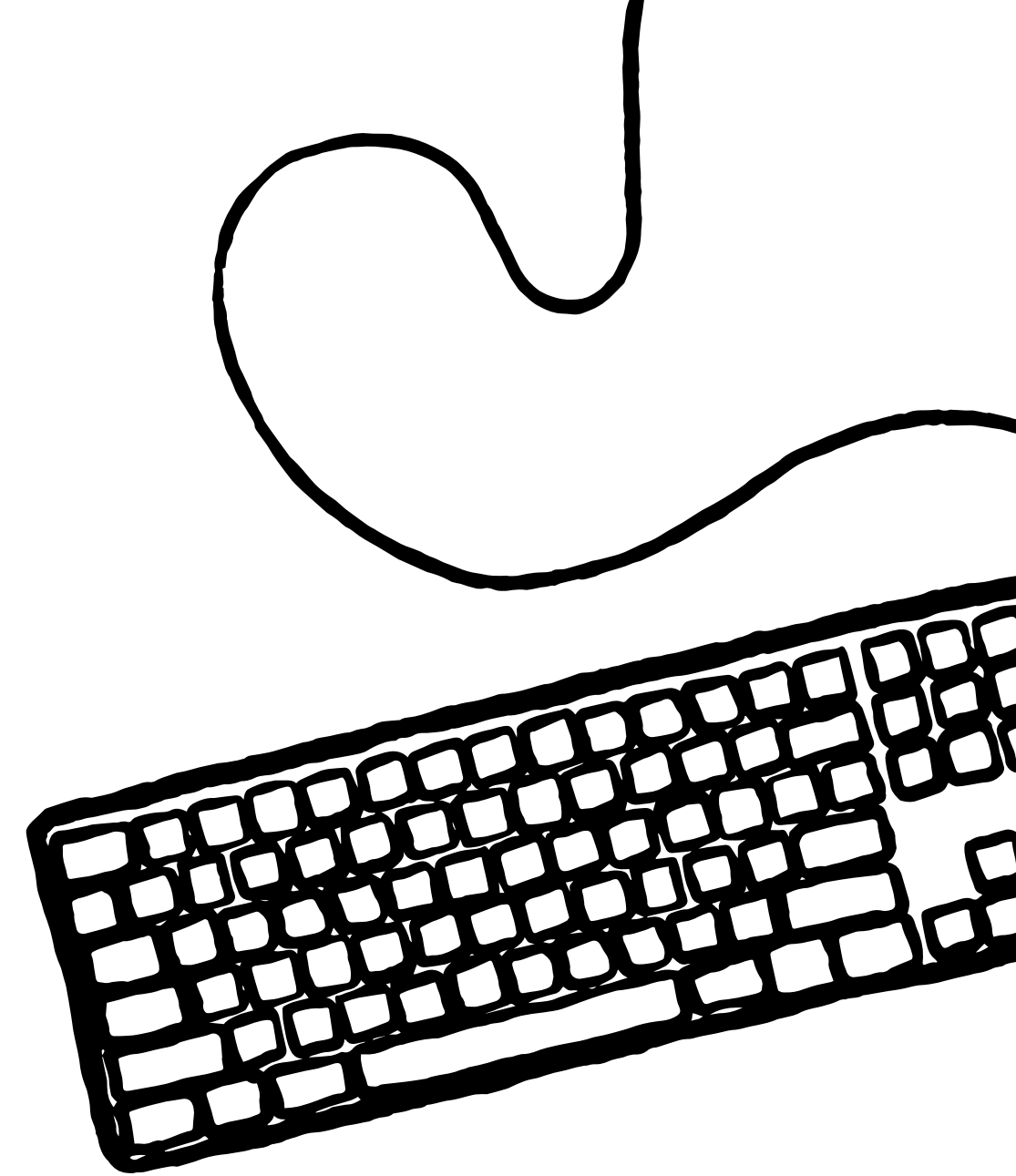
7 Intro

You will learn how to set up, implement, and manage the routing system in your app with Vue Router.

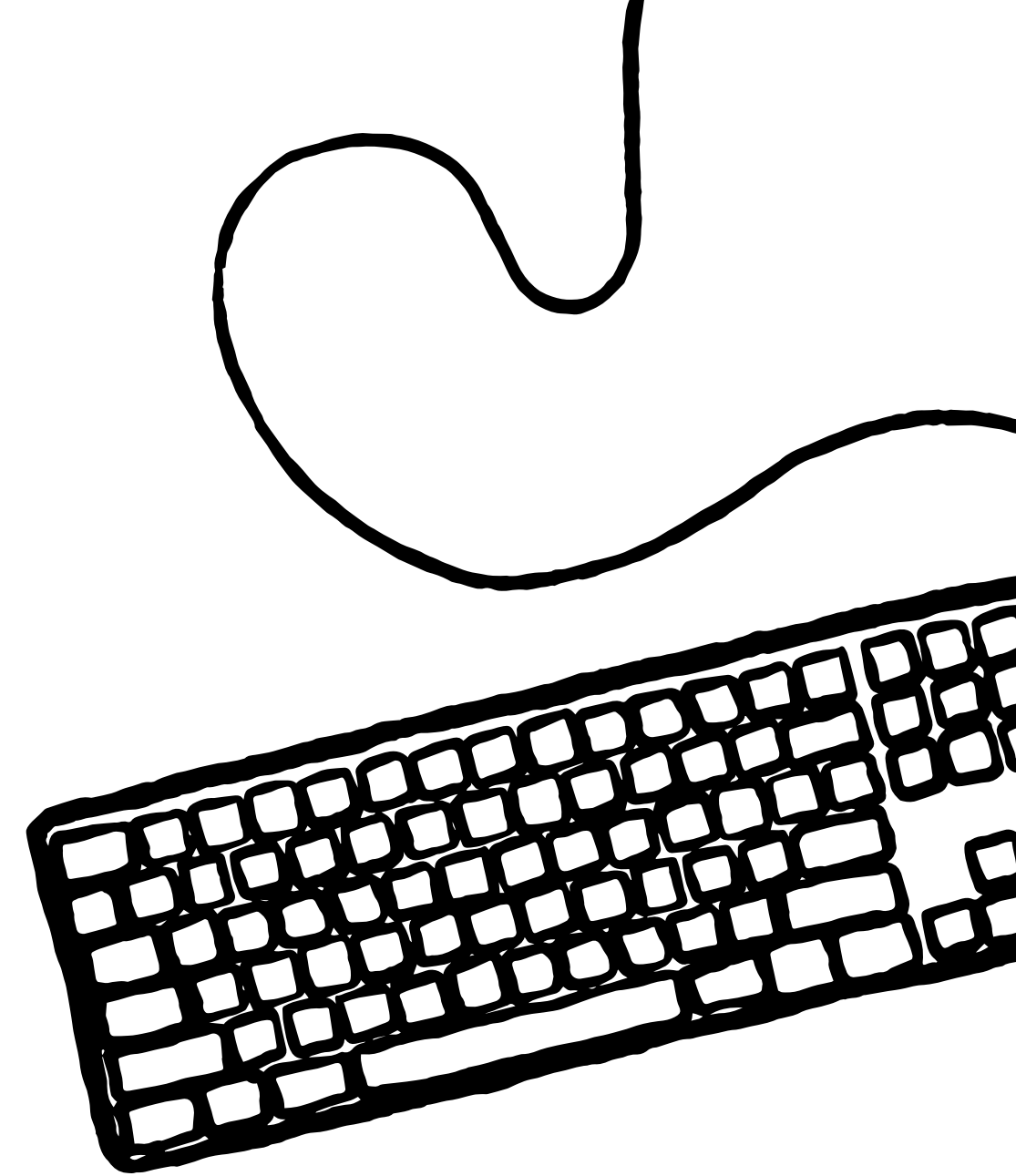
You will look at dynamic routing for passing parameter values and nested routes for better reusability in complex applications.

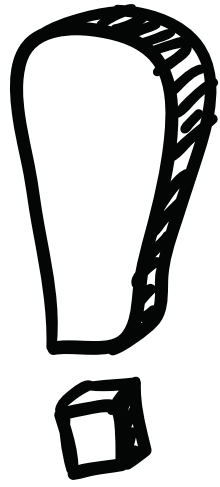
- Understanding routing
- Understanding Vue Router
- Exploring the RouterView element
- Defining the routes
- Setting up a default layout for your app
- Setting up navigation links with RouterLink
- Passing route parameters
- Understanding Router Hooks
- Decoupling params with props
- Dynamic routing
- Catching error paths
- Nested routes
- Using layouts

- **Routing** is one of the most essential and primary parts of building dynamic web applications.
- It is the process of getting a user to their desired location. Users who enter `website.com/about` into their URL bar will be redirected to the About page.
- In web development, routing is the matching mechanism by which we decide how to connect HTTP requests to the code that handles them.
- We use routing whenever there is a need for URL navigation in our application. Most modern web applications contain a lot of different URLs, even single-page ones.
- Thus, routing creates a navigation system and helps users quickly move around our application and the web. With Single-Page Applications (SPAs), routing allows you to smoothly navigate within an application without the need for page refreshing.



In short, routing is a way for an application to interpret what resource users want based on the URL provided. It is a system for web-based resource navigation with URLs, such as paths to assets (images and videos), scripts, and stylesheets.





VueRouter

VueRouter is the official router service for any Vue.js application.

It provides a single entry point for communication between components through routes, effectively controlling the application's flow regardless of the user's behavior.

```
npm init vue@3
```

Vue.js - The Progressive JavaScript Framework

```
✓ Project name:  chapter-3-vue-app
✓ Add TypeScript?  No    Yes
✓ Add JSX Support?  No    Yes
✓ Add Vue Router for Single Page Application development?  No  Yes
✓ Add Pinia for state management?  No  Yes
✓ Add Vitest for Unit Testing?  No  Yes
✓ Add Cypress for End-to-End testing?  No    Yes
✓ Add ESLint for code quality?  No  Yes
✓ Add Prettier for code formatting?  No  Yes
```

If you would like to add Vue Router to an existing Vue.js application, you can install it as an application's dependency with the following command:

```
npm install vue-router
```

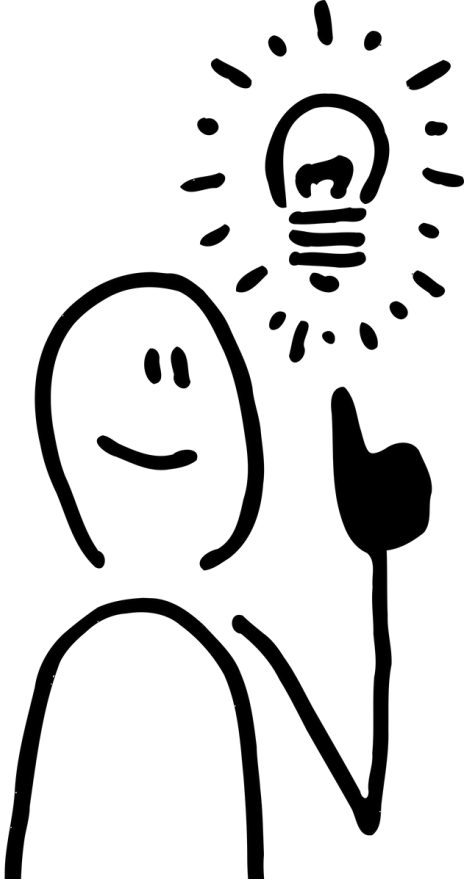
Exploring the RouterView element

RouterView is a Vue component whose job is to do the following:

- Render different child components
- Mount and unmount itself automatically at any nesting level, depending on the route's given path

Without RouterView, it is almost impossible to render dynamic content correctly for users at runtime.

For example, when a user navigates to the Home page, RouterView knows and only generates the content related to that page.



Passing props to view

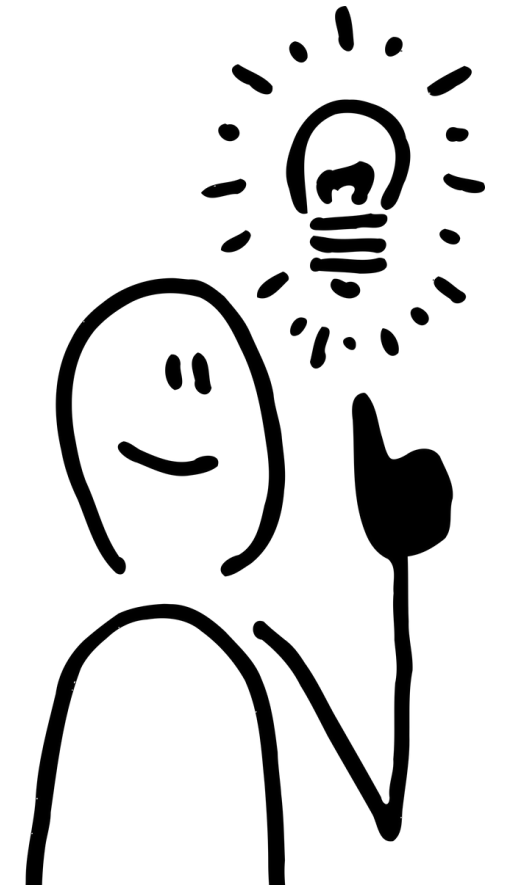
RouterView is a component → it can receive props.
The only prop it receives is **name** → the same name registered in the corresponding route's record defined in the **router object** at the initialization phase.

```
<RouterView class="main-app-view"/>
```

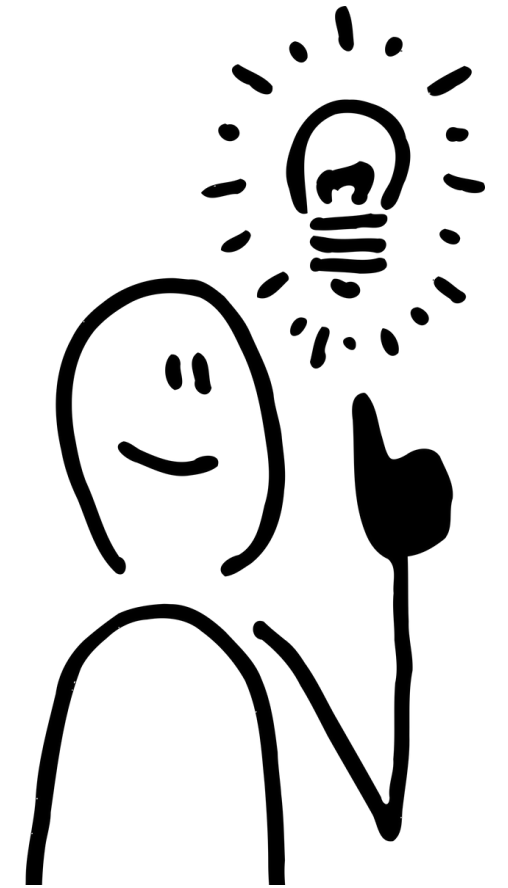
```
<template>  
<div>Hello World</div>  
</template>
```

output:

```
<div class="main-app-view">Hello World</div>
```



Working with RouterView



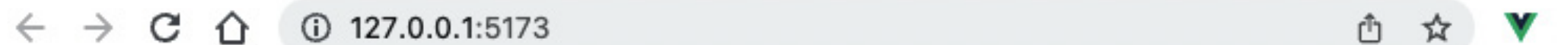
App.vue

```
<template>
  <div id="app">
    <RouterView/>
  </div>
</template>
```

src/router/index.js, comment out the generated code within the routes array:

```
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    // {
    //   path: '/',
    //   name: 'home',
    //   component: HomeView
    // },
    // {
    //   path: '/about',
    //   name: 'about',
    //   component: () => import('../views/AboutView.vue')
    // }
  ]
})
```

output:



Setting up Vue Router

When we add Vue Router to our project, Vite creates and adds a router folder to the /src directory with a single auto-generated index.js file.

src/main.js

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
const app = createApp(App)
app.use(router)
```

app.use is an instance method with a built-in mechanism to prevent you from installing a plugin more than once.

You can import **useRoute()** and **useRouter()** functions from **vue-router** package and get the **current route object** (instead of this.\$route), and **global router object** (instead of this.\$router)

```
<script setup>
import { useRoute, useRouter } from 'vue-router'
const route = useRoute();
const router = useRouter();
// component's logic...
</script>
```

After executing app.use(router), the following objects are available for access in any component:

- **this.\$router**: The global router object
- **this.\$route**: The current route object points to the element in context



Defining the routes

Each route is an object literal that uses the `RouteRecordRaw` interface with the following properties:

```
interface = {  
  path: string,  
  component?: Component,  
  name?: string, // for named routes  
  components?: { [name: string]: Component }, // for named views  
  redirect?: string | Location | Function,  
  props?: boolean | Object | Function,  
  alias?: string | Array<string>,  
  children?: Array<RouteConfig>, // for nested routes  
  beforeEnter?: (to: Route, from: Route, next: Function) => void,  
  meta?: any,  
  sensitive?: Boolean,  
  strict?: Boolean  
}
```



Defining the routes

We define all the routes needed for an application as a list of routes

```
const routes = [  
  //Route1,  
  //Route2,  
  //...  
]
```

The next property is **name**, even though it is optional, we recommend defining every path with a name for better code maintenance and route tracking purposes

src/router/index.js: uncomment the code inside routes.
For convenience, there will be two predefined routes called home and about, each being an object located in the routes array.

```
{  
  path: '/',  
  name: 'home',  
  component: HomeView  
}
```

required

the **/about** path will be translated into <app domain>/about (localhost:8080/about or example.com/about)

component property, is a Vue component instance. RouterView uses this property to reference the view component to render the page content when the path is active.



Setting up the router instance

After defining the routes, the final step is to create the router instance based on the given configuration options by using the `createRouter` method

```
import { createRouter, createWebHistory } from 'vue-router'
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL), routes
})
```

routes

a required option, without this, the router won't be able to recognize the paths and direct users to the suitable view content accordingly.

history

history determines the router's mode:

- *HTML History mode*: It allows us to achieve URL navigation without a page reload and makes the URL path human-readable – for example, **yourapplication.com/about**.
- *Hash mode*: for example, **yourapplication.com/#about** for an About page or **yourapplication.com/#/** for the Home URL of your app.

base

determines the base URL for the app. For example, when we set it to **process.env.BASE_URL**, it allows developers to control the base URL from outside the application code, specifically from a `.env` file.



Setting up the router instance

With base finally out of the way, we have created the router instance. All that is left is to export it:

```
export default router
```

App.vue

```
<template>
<header>
  
  <div class="wrapper">
    <HelloWorld msg="You did it!" />
  </div>
</header>
<RouterView />
</template>
```

main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router'
const app = createApp(App)
app.use(router)
app.mount('#app')
```



You did it!

You've successfully created a project with
Vite + Vue 3.

Documentation

Vue's [official documentation](#) provides you with all information you need to get started.

Tooling

This project is served and bundled with [Vite](#). The recommended IDE setup is [VSCode](#) + [Volar](#). If you need to test your components and web pages, check out [Cypress](#) and [Cypress Component Testing](#). More instructions are available in [README.md](#).

Ecosystem

Get official tools and libraries for your project: [Pinia](#), [Vue Router](#), [Vue Test Utils](#), and [Vue Dev Tools](#). If you need more resources, we suggest paying [Awesome Vue](#) a visit.

Community

Got stuck? Ask your question on [Vue Land](#), our official Discord server, or [StackOverflow](#). You should also subscribe to [our mailing list](#) and follow the official [@vuejs](#) twitter account for latest news in the Vue world.

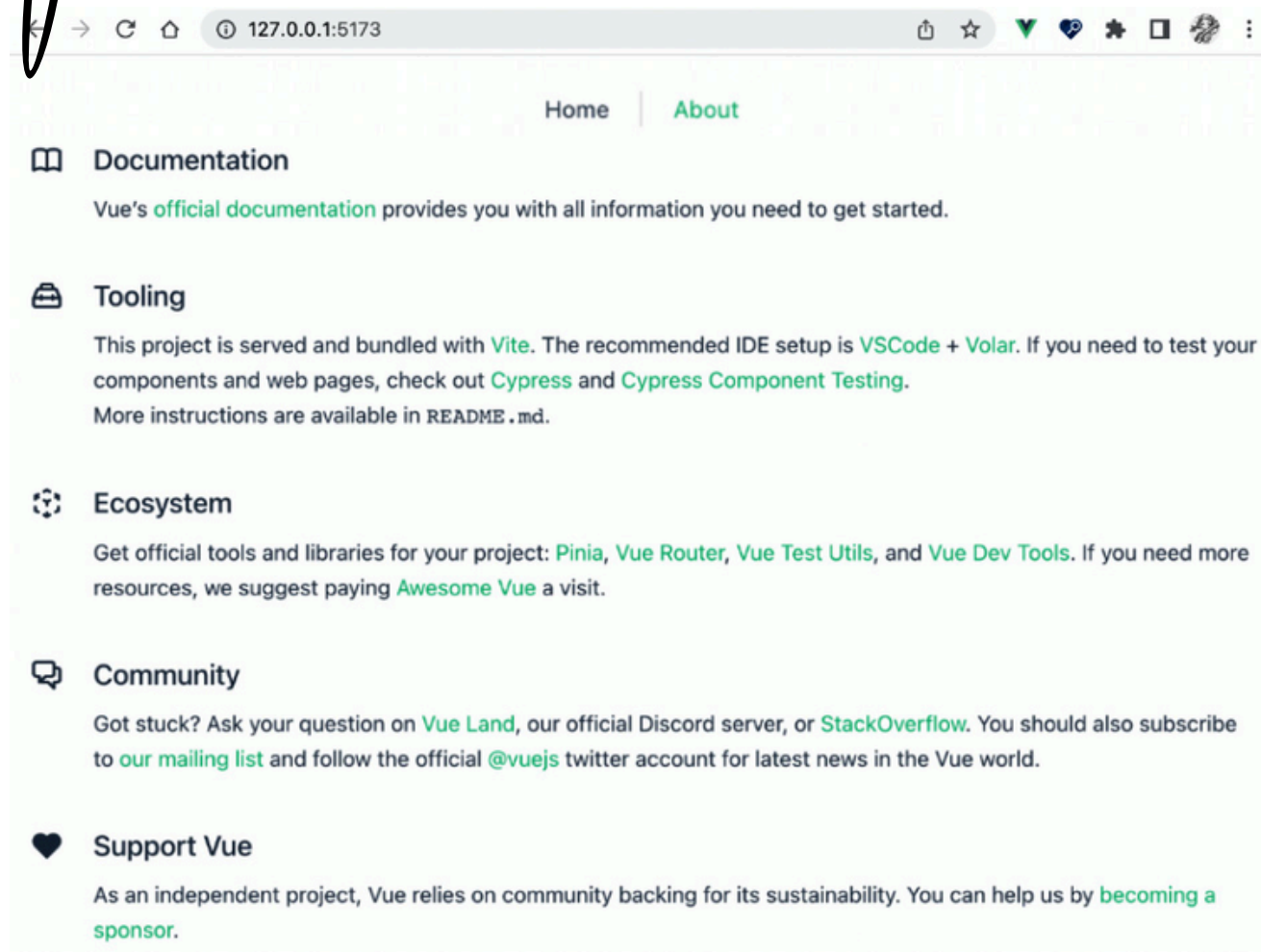
Support Vue

As an independent project, Vue relies on community backing for its sustainability. You can help us by [becoming a sponsor](#).

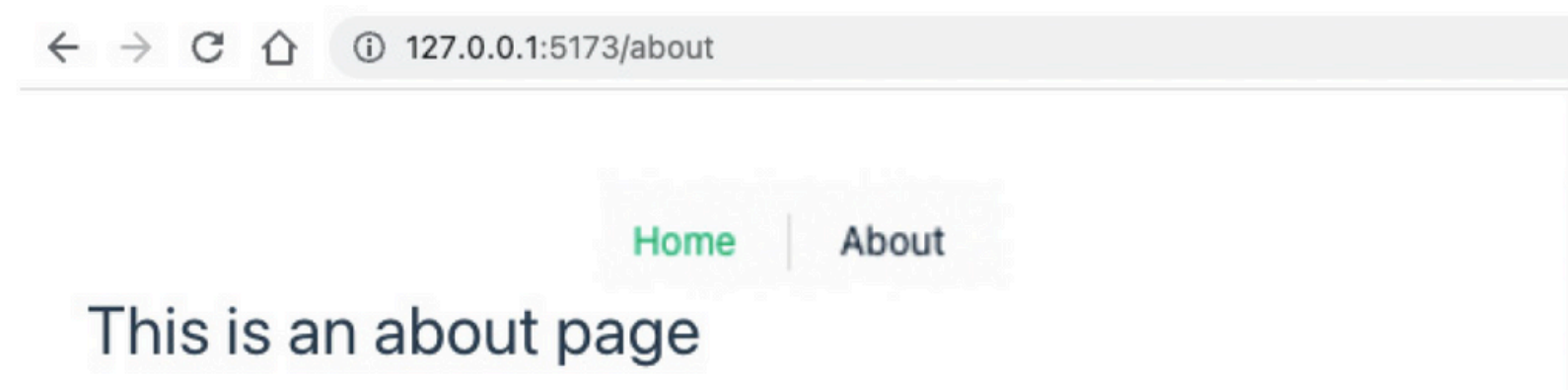
Setting up a default layout for your app

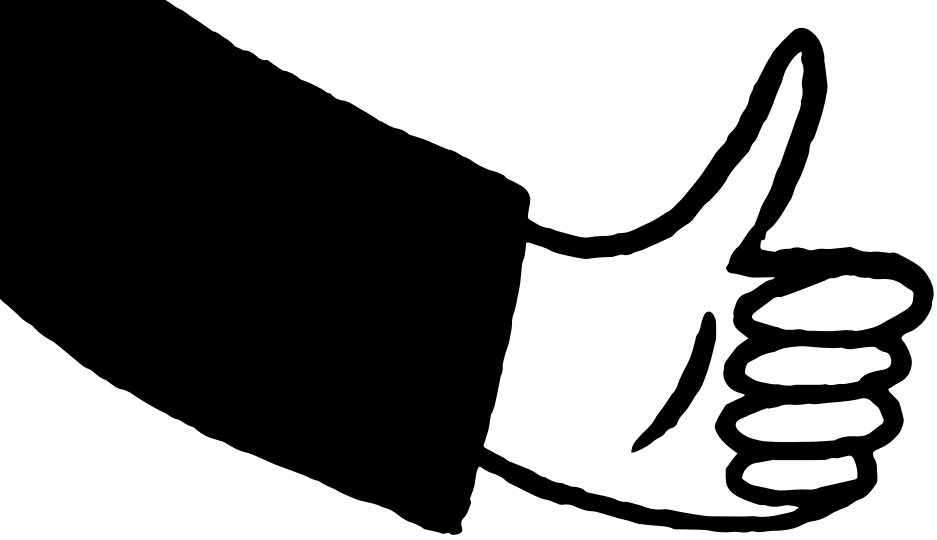
the content changes between pages while the header menu stays the same.

```
<template>
  <header>
    <nav>
      <RouterLink to="/">Home</RouterLink>
      <RouterLink to="/about">About</RouterLink>
    </nav>
  </header>
  <RouterView />
</template>
```



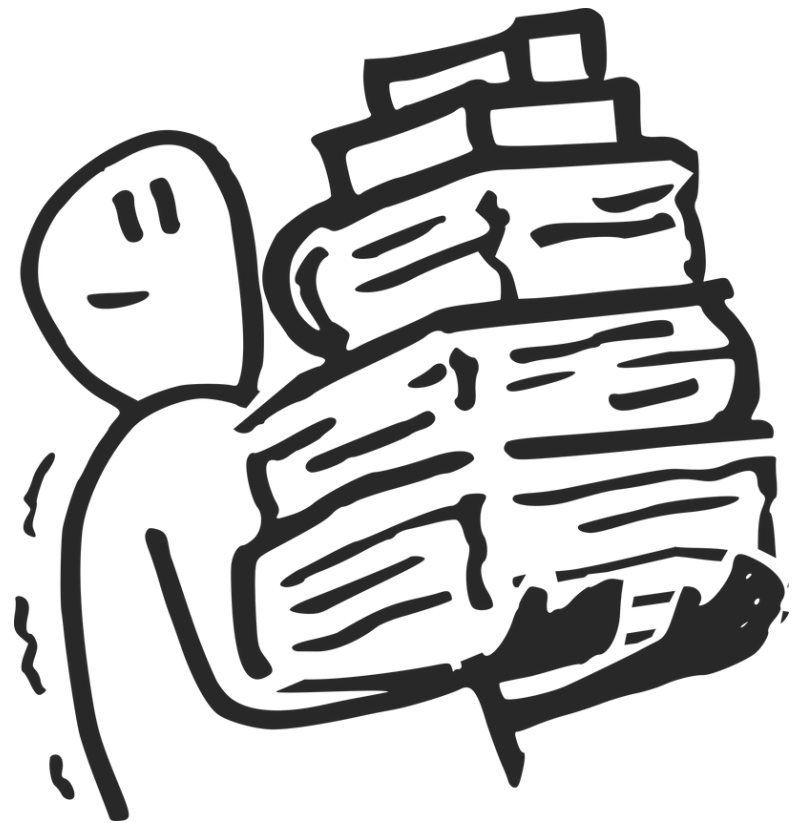
Once you navigate to the /about page, the header links will not change, while the content now become the following:





Exercise 7.01 – implementing a Message Feed page using Vue Router





Setting up navigation links with RouterLink

- **RouterView** oversees rendering the correct active view content relative to the URL path;
- **RouterLink** oversees mapping the routes to navigable links.

RouterLink by default renders an anchor tag, `<a>`, with a valid `href` link generated by its `to` prop.

```
<nav>
  <RouterLink to="/">Home</RouterLink>
  <RouterLink to="/about">About</RouterLink>
</nav>
```

Since we are using the web history mode with `createWebHistory()`, the `to` prop of each **RouterLink** should receive an identical value with the `path` property declared in the targeted route object (as in the list of routes defined in `src/router/index.js`).

An alternative way of using the `to` prop is to bind it with an object containing the route name instead of the path.

```
<nav>
  <RouterLink :to="{ name: 'home' }">Home</RouterLink>
  <RouterLink :to="{ name: 'about' }">About</RouterLink>
</nav>
```

```
▼ <nav data-v-7a7a37b1>
  <a href="/" class="router-link-active router-link-exact-active" data-v-7a7a37b1
    aria-current="page">Home</a> == $0
  <a href="/about" class data-v-7a7a37b1>About</a>
</nav>
```

Note that since we have access to **this.\$router** within a component, we can trigger a navigation route programmatically by using **this.\$router.push()** and pass a path or a router object in a similar way to using to:

```
this.$router.push('/home')
```

 or:

```
import { useRouter } from 'vue-router'
const router = useRouter();
router.push('/home');
```



Sometimes, we want to navigate back to the previous page. Using **this.\$router.push()** can achieve this, but it adds more routes in the history stack instead of going back.

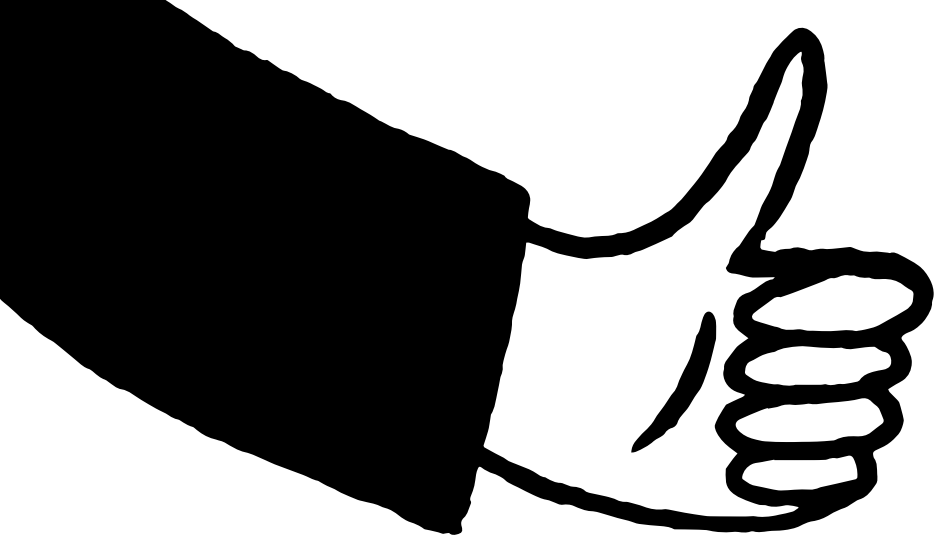
The correct technique is to use **this.\$router.go(steps)**, in which **steps** is an integer that indicates the number of steps to go back or forward in the history stack. This functionality works similarly to **window.history.go(steps)**.

```
this.$router.go(-1)
// similar to window.history.back() - go back one page
```

```
this.$router.go(1)
// similar to window.history.forward() -go forward one page
```

or:

```
import { useRouter } from 'vue-router'
const router = useRouter();
router.go(1); //forward one page
router.go(-1); //back one page
```



Exercise 7.02 – adding the navigation link to the MessageFeed route

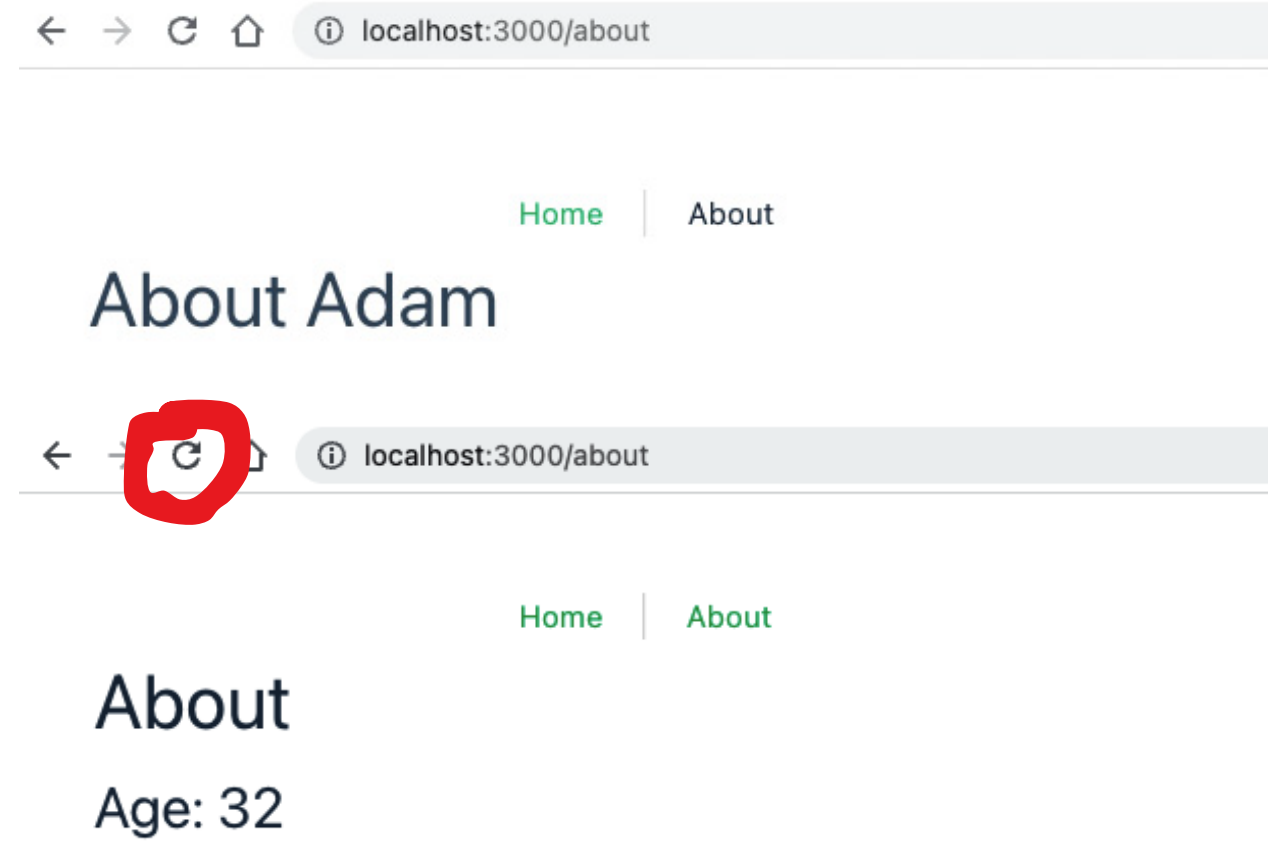


Passing route parameters

In our example app, we want our about page to be able to receive a data string called user, as the user's name from the link triggered.

Before Vue Router 4.1.4, we could achieve this feature by changing the to prop from a string literal to an object literal with a name and params properties:

```
<RouterLink :to="{ name: 'about', params: { user: 'Adam' }}">  
  About  
</RouterLink>
```



Passing route parameters

Query params for a URL route starts after the question mark - ?

```
<your-app-url>?<param1>=<value1>&<param2>=<value2>
```

```
localhost:3000/about?user=Adam
```

And in the About component, we will retrieve the query field from the route object, and get the relevant field's value

```
<script setup>
import { useRoute } from 'vue-router'
const route = useRoute();
const { user } = route.query;
</script>
```

In the template section, we can replace `$route.params.user` to `user`, and the output stays the same, even on refreshing the page

```
<template>
  <div class="about">
    <h1>About {{ user }}</h1>
  </div>
</template>
```

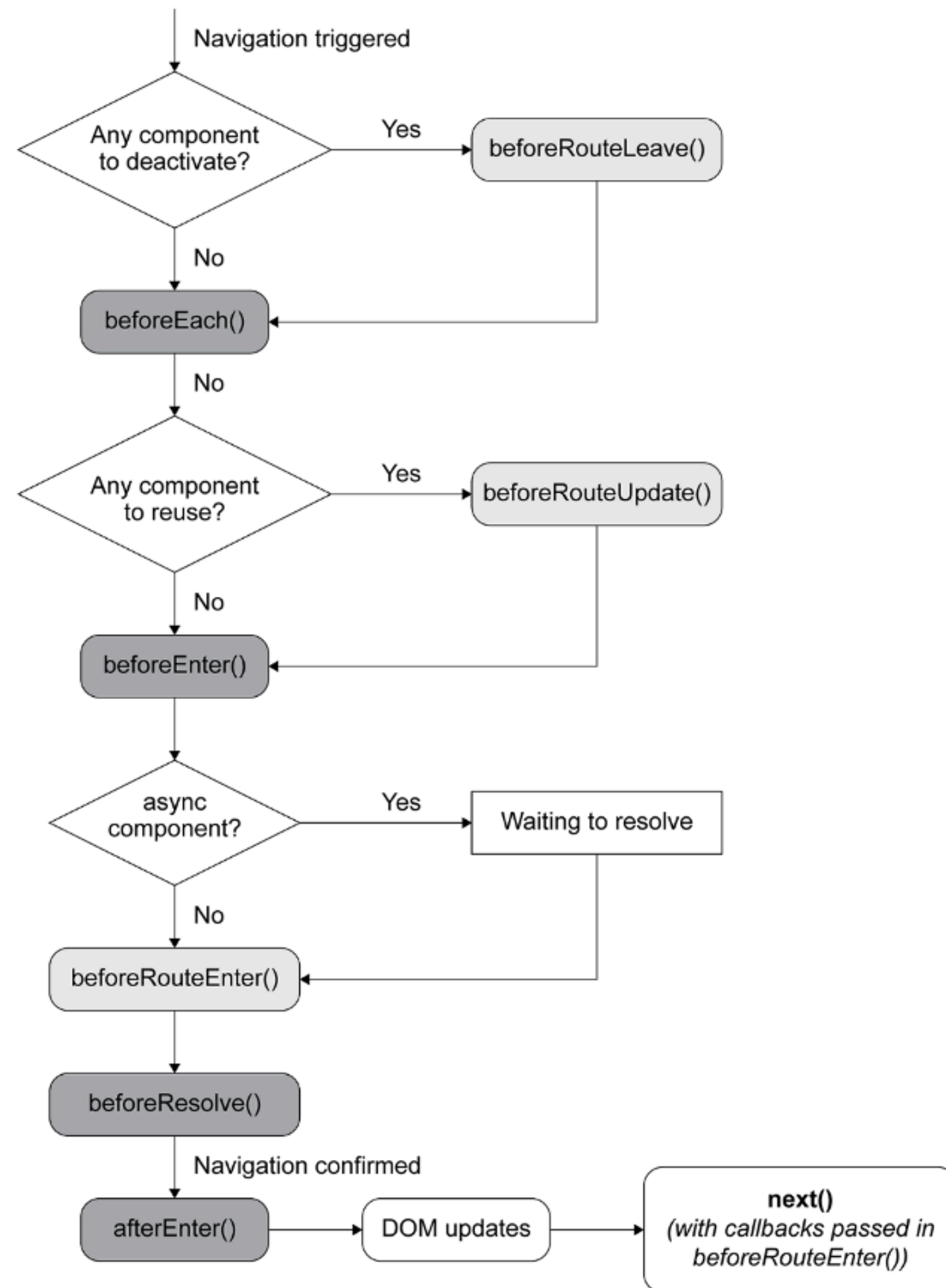
Understanding Router Hooks

Once navigation is triggered for a certain route, Vue Router provides several primary navigation guards, or Hooks, for developers to guard or intercept that navigation process.

These guards can be hooked either globally or in the component, depending on the type.

Some examples are as follows:

- Globally: `beforeEach`, `beforeResolve`, and `afterEach`
- Per component: `beforeEnter`
- In-component: `beforeRouteUpdate`, `beforeRouteEnter`, and `beforeRouteLeave`



Setting up `beforeEach` Hooks

`beforeEach` is a global Hook and is called at the very beginning of navigation, before the other global and in-component Hooks are triggered (except for `beforeRouteLeave` in the previous view component). It should be defined as a global method of the router instance during initialization in the `index.js` file and takes the following syntax

```
const router = createRouter({
  //...
})
router.beforeEach(beforeEachCallback)
```

```
const beforeEachCallback = (
  to, // The destination route
  from, //The source route
  next //The function to trigger to resolve the hook
) => { ... }
```

or:

```
router.beforeEach((to, from, next) => { ... })
```



Distinguishing between the **beforeEach** and **beforeResolve** Hooks

We can also register a global Hook with **beforeResolve** with the same syntax. Unlike **beforeEach**, which is triggered when navigation is set up, **beforeResolve** will be triggered just before the navigation is carried out and confirmed after all the Hooks (both global and in-component) are resolved:

```
router.beforeResolve((
  to, // The destination route
  from, //The source route
  next //The function to trigger to resolve the hook
) => {
  if (to.name === 'about' && (!to.query?.user)) {
    next({ name: 'error' })
  }
  else {
    next();
  }
})
```



The afterEach Hook

The **afterEach()** Hook is the **last global** navigation guard to be triggered after the navigation is confirmed (which means after **beforeResolve()**).

Unlike the other global guards, the hook function to pass to **afterEach()** does not receive a next function – hence, it won't affect the navigation.

In addition, the **to** and **from** parameters are **read-only** Route objects.

Hence, the best use case for **afterEach** is to save data such as the last visited Route object for a Back button, the passed parameters of the route destination, or page view tracking.

```
let user = 'Adam';
router.beforeEach((to, from, next) => {
  if (to.name === 'about' && (!to.query?.user)) {
    next({ name: 'about', query: { user } })
  }
  else {
    user = to.query.user;
    next()
  }
});
router.afterEach((to, from) => {
  if (to.name === 'about' && to.query && to.query.user) {
    user = to.query.user;
  }
})
```

App.vue

```
<RouterLink to="/about?user=Alex"> About </RouterLink>
```

← → ↺ 🏠 ⓘ 127.0.0.1:3000/about?user=Alex

Home | About

About Alex

← → ↺ 🏠 ⓘ 127.0.0.1:3000/about?user=Adam

Home | About

About Adam



We can have the about component now with the `beforeRouteEnter()` Hook defined as follows

```
<script>
export default {
  data() {
    return {
      user: ''
    }
  },
  beforeRouteEnter(to, from, next) {
    if (!to.params || !to.params.user) {
      next(comp => {
        comp.user = 'Alex'
      })
    }
    else {
      next();
    }
  }
}
</script>
```

As you can see, we don't have access to the **this** scope of the component during **beforeRouteEnter** because the view component is still being created.

Luckily, we can access the instance through a callback passed to **next()**.

Whenever the navigation is confirmed, which means the component is created, the callback will be triggered, and the component instance will be available as the sole argument (comp) of the callback.

```
import { useRoute } from 'vue-router';
import { ref } from 'vue';

const user = ref('');
const route = useRoute();
if (!route.params || !route.params.user) {
  user.value = 'Alex'
}
```

For `beforeRouteUpdate` and `beforeRouteLeave`, the component has been created – hence, this instance is available and there is no need for a callback for `next()`. In fact, a callback function is only supported in `next()` within the use of `beforeRouteEnter()`

Setting up in-component Hooks

By setting this property's value as a function which accepts a route and returns an object containing an user field of value based on route.query.user, the router will automatically understand and map any route.query parameters into the props of the view component accordingly

index.js

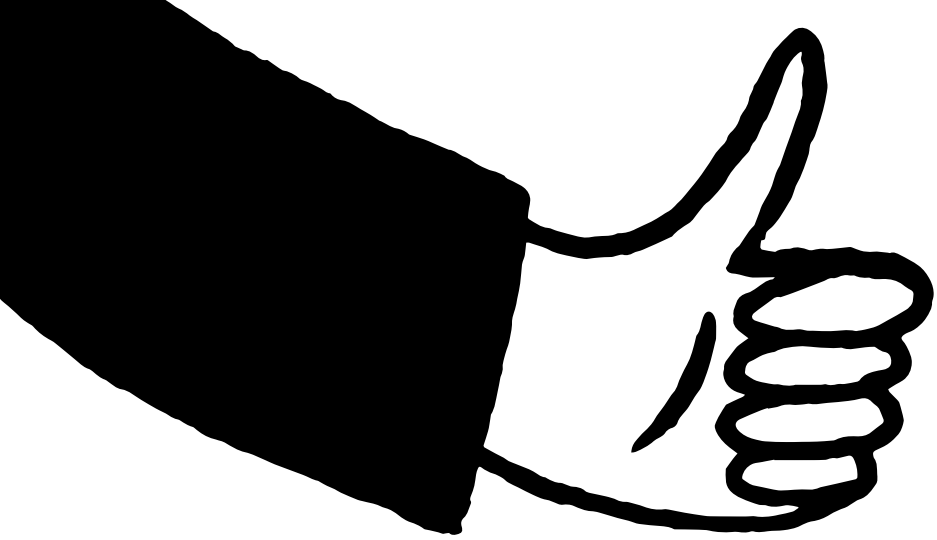
```
{
  path: '/about',
  name: 'about',
  component: () => import('../views/AboutView.vue'),
  props: route => ({ user: route.query.user || 'Adam' })
}
```

AboutView.vue

```
<script setup>
import { defineProps } from 'vue'
const props = defineProps({
  user: String
})
</script>
```

```
<template>
  <div class="about">
    <h1>About {{user}}</h1>
  </div>
</template>
```

Decoupling Params with Props



Exercise 7.03:
**Passing the content of the
selected message to a
new message page and having it
printed out**



Dynamic routing

For example, we want to render the User view component for every user but with different id values. Vue Router provides us with the ability to use dynamic segments denoted by a colon (:) to achieve dynamic routing.

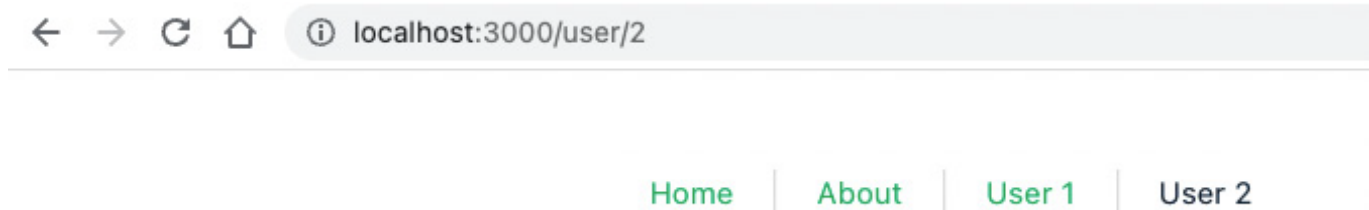
Instead of using params, which doesn't persist its value when you refresh the page or appear in the URL, we define the required params directly in the path as follows

```
{
  path: '/user/:id',
  name: 'user',
  component: () => import('../views/User.vue'),
  props: true,
}
```

:id means params here is not static

```
<template>
  <div>
    <h1>About a user: {{$route.params.id}}</h1>
  </div>
</template>
```

The value of :id will be exposed as \$route.params.id in that view component's instance



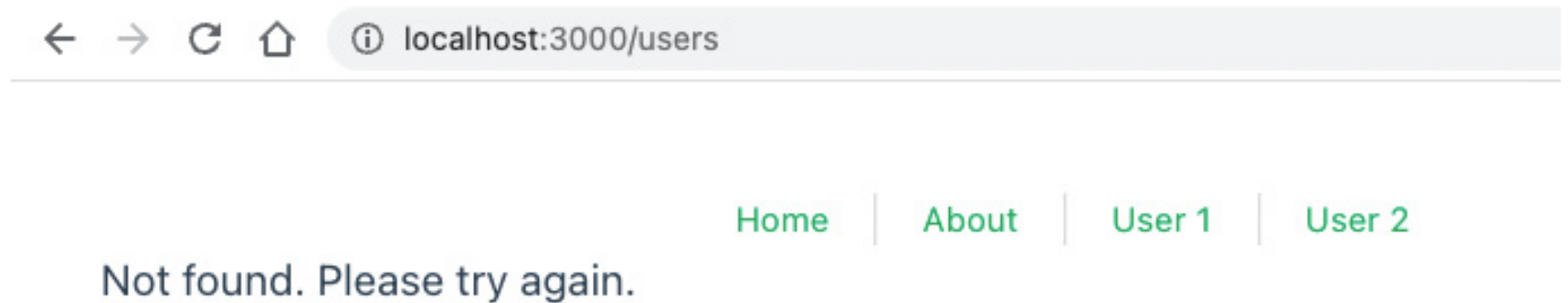
About a user: 2

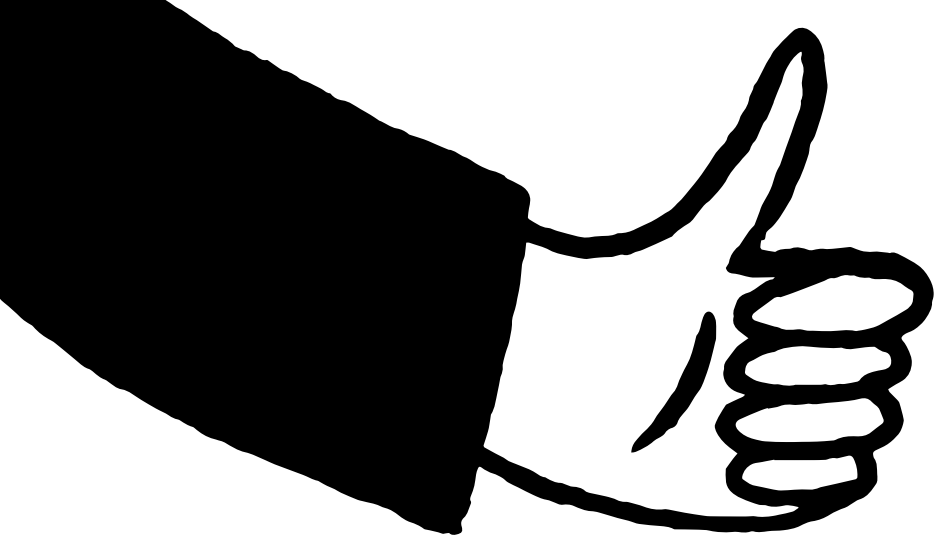
Catching error paths

Other important routes that we always need to remember to handle besides the **Home page** ('/') include **error routes**, such as **404 Not found** when the URL path doesn't match any registered path, among others.

For 404 Not found, we can use the Regex pattern, **`/:pathMatch(.*)*`**, which stands for matching every other URLs, to collect all the cases that don't match the defined routes.

```
{
  path: '/:pathMatch(.*)*',
  name: '404',
  component: () => import('../views/404.vue'),
}
```





Exercise 7.04 – implementing a message route for each message with the dynamic routing pattern



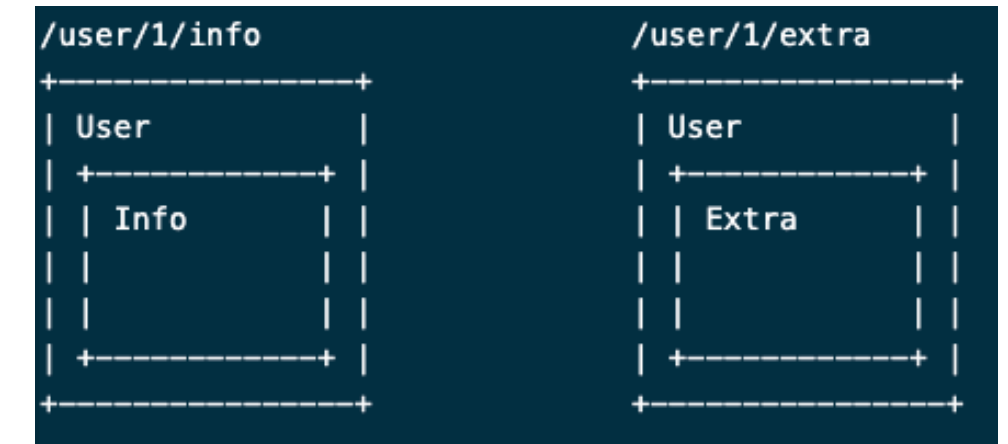
For example, /user/settings/general indicates that a general view is nested in the settings view and this settings view is nested within the user view. It represents the General information section of a user's settings page.

User.vue

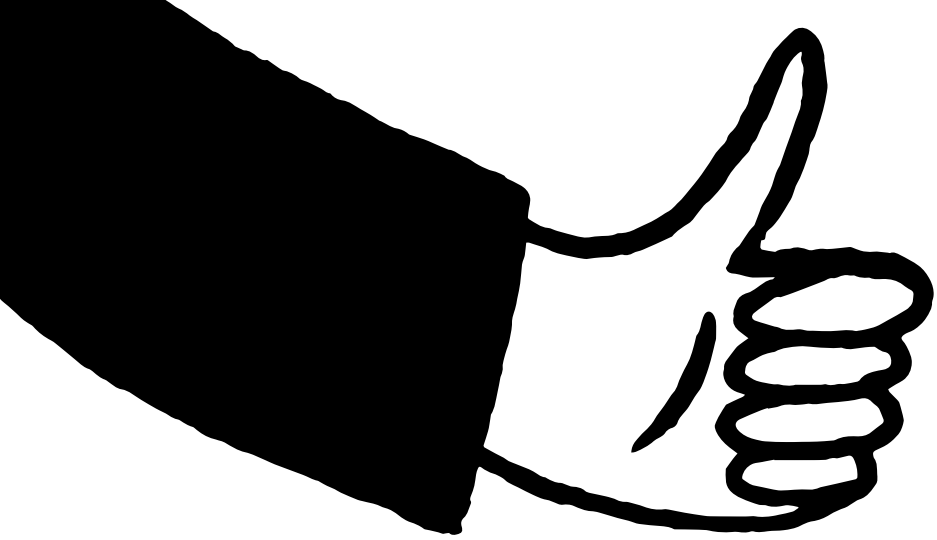
```
<div>
  <h1>About a user: {{$route.params.id}}</h1>
  <RouterLink :to="`/user/${$route.params.id}/info`">
    Info
  </RouterLink> |
  <RouterLink :to="`/user/${$route.params.id}/extra`">
    Extra
  </RouterLink>
  <RouterView />
</div>
```

index.js

```
{
  path: '/user/:id',
  name: 'user',
  component: () => import('../views/User.vue'),
  props: true,
  children: [{
    path: 'info',
    name: 'userinfo',
    component: () => import('../views/UserInfo.vue'),
    props: true,
  }, {
    path: 'extra',
    component: () => import('../views/UserExtra.vue')
  }]
}
```



Nested routes



Exercise 7.05 – building navigation tabs within the message view



Questions?

